

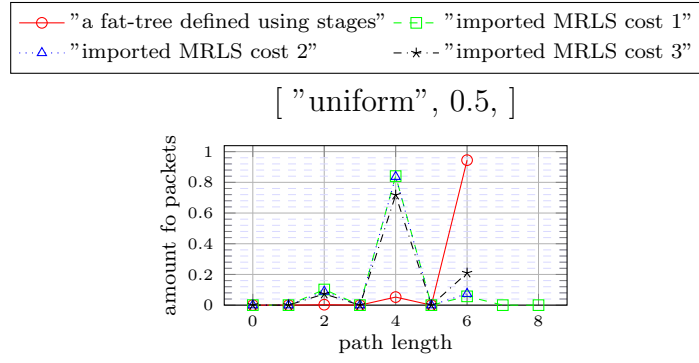


Figure 1: X: ["uniform", 0.5,]

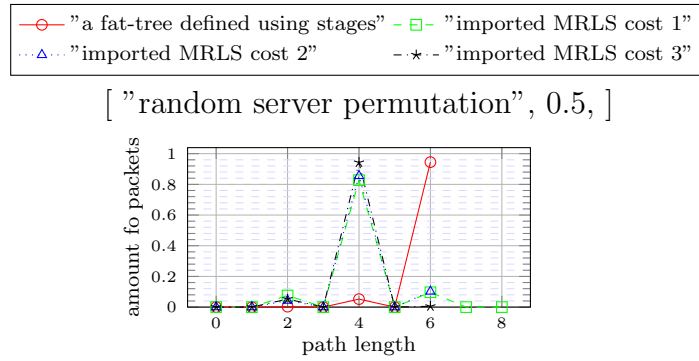


Figure 2: X: ["random server permutation", 0.5,]

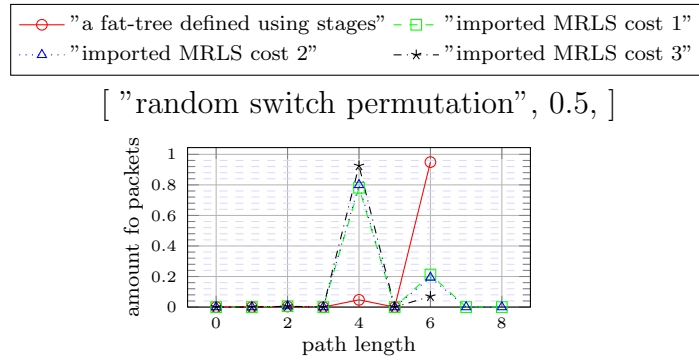


Figure 3: X: ["random switch permutation", 0.5,]

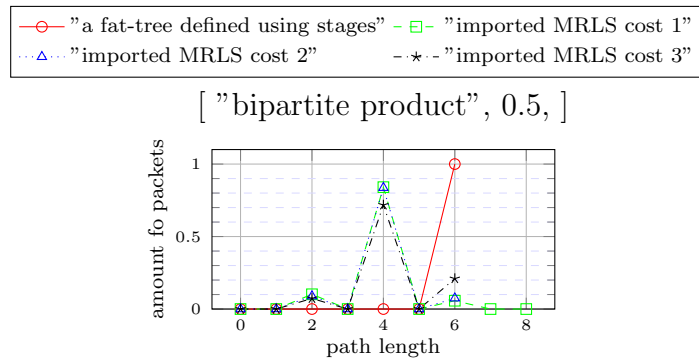


Figure 4: X: ["bipartite product", 0.5,]

The following versions used in the simulations.

- heads/alex-stable-0ddd70b98d380a94f1ddc39d4b3992e370e55d3d(0.6.3)
- heads/alex-dev-ef489f87de796fb3ec38c1bb4368719b2cb116eb(0.6.3)
- heads/alex-dev-7775ae14e7573de7f634e7a6d77d154c320a8b9a(0.6.3)

```
Configuration{
  random_seed: ![ 42, 935, 128643 ],
  warmup: 20000,
  measured: 50000,
  general_frequency_divisor: 2,
  topology: topo![
    MultiStage{
      stages: [
        Fat{ bottom_factor: 18, top_factor: 18 },
        Fat{ bottom_factor: 18, top_factor: 18 },
        Fat{ bottom_factor: 18, top_factor: 18 }],
      servers_per_leaf: 18,
      legend_name: "a fat-tree defined using stages"},
    MultiStage{
      stages: [
        ExplicitStageFile{ filename: "../MRLS_5832_18_2916_36" }],
      servers_per_leaf: 18,
      legend_name: "imported MRLS cost 1"},
    MultiStage{
      stages: [
        ExplicitStageFile{ filename: "../MRLS_8748_24_5832_36" }],
      servers_per_leaf: 12,
      legend_name: "imported MRLS cost 2"},
    MultiStage{
      stages: [
        ExplicitStageFile{ filename: "../MRLS_11664_27_8748_36" }],
      servers_per_leaf: 9,
      legend_name: "imported MRLS cost 3"}],
  traffic: HomogeneousTraffic{
    servers: 104976,
    pattern: ![
      Uniform{ legend_name: "uniform" },
      RandomPermutation{ legend_name: "random server permutation" },
      CartesianTransform{
        sides: [ 18, 5832 ],
        patterns: [ Identity, RandomPermutation ],
        legend_name: "random switch permutation"},
      Product{
        block_size: 52488,
        block_pattern: Uniform,
        global_pattern: CartesianTransform{
          sides: [ 2 ],
          shift: [ 1 ],
          legend_name: ""},
        legend_name: "bipartite product"}],
    load: ![ 0.5 ],
    message_size: 16},
  maximum_packet_size: 16,
  router: InputOutput{
    virtual_channels: topo![ 3, 4, 4, 4 ],
    virtual_channel_policies: topo![
      [
        OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
        use_neighbour_space: true, aggregate: true },
        OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
        use_neighbour_space: true, aggregate: false },
        LowestLabel,
        EnforceFlowControl,
        Random],
      [
        MapHop{
          hop_to_policy: [
            ArgumentVC{
              allowed: [ 0 ]},
            ArgumentVC{
              allowed: [ 0 ]},
            ArgumentVC{
              allowed: [ 1 ]},
            ArgumentVC{
              allowed: [ 1 ]},
            ArgumentVC{
              allowed: [ 2 ]},
            ArgumentVC{
              allowed: [ 2 ]},
            ArgumentVC{
              allowed: [ 3 ]},
            ArgumentVC{
              allowed: [ 3 ]}],
          VecLabel{
            label_vector: [ 0, 64, 80 ]},
            OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
            use_neighbour_space: true, aggregate: true },
            OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
            use_neighbour_space: true, aggregate: false },
            LowestLabel,
            EnforceFlowControl,
            Random],
          [
            MapHop{
              hop_to_policy: [
                ArgumentVC{
                  allowed: [ 0 ]},
                ArgumentVC{
                  allowed: [ 0 ]},
                ArgumentVC{
                  allowed: [ 1 ]},
```

```

    ArgumentVC{
      allowed: [ 1 ]},
    ArgumentVC{
      allowed: [ 2 ]},
    ArgumentVC{
      allowed: [ 2 ]},
    ArgumentVC{
      allowed: [ 3 ]},
    ArgumentVC{
      allowed: [ 3 ]}],
  VecLabel{
    label_vector: [ 0, 64, 80 ]},
  OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
use_neighbour_space: true, aggregate: true },
  OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
use_neighbour_space: true, aggregate: false },
  LowestLabel,
  EnforceFlowControl,
  Random],
[
  MapHop{
    hop_to_policy: [
      ArgumentVC{
        allowed: [ 0 ]},
      ArgumentVC{
        allowed: [ 0 ]},
      ArgumentVC{
        allowed: [ 1 ]},
      ArgumentVC{
        allowed: [ 1 ]},
      ArgumentVC{
        allowed: [ 2 ]},
      ArgumentVC{
        allowed: [ 2 ]},
      ArgumentVC{
        allowed: [ 3 ]},
      ArgumentVC{
        allowed: [ 3 ]}],
    VecLabel{
      label_vector: [ 0, 64, 80 ]},
    OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
use_neighbour_space: true, aggregate: true },
    OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
use_neighbour_space: true, aggregate: false },
    LowestLabel,
    EnforceFlowControl,
    Random]],
  allocator: Random,
  buffer_size: 128,
  bubble: false,
  flit_size: 16,
  intransit_priority: false,
  allow_request_busy_port: true,
  output_buffer_size: 64,
  crossbar_frequency_divisor: 1,
  crossbar_delay: 2},
routing: topo![
  UpDown{ legend_name: "minimal routing" },
  Polarized{ include_labels: true, legend_name: "Polarized original" },
  Polarized{ include_labels: true, legend_name: "Polarized original" },
  Polarized{ include_labels: true, legend_name: "Polarized original" }],
link_classes: [
  LinkClass{ delay: 2 },
  LinkClass{ delay: 2 },
  LinkClass{ delay: 2 },
  LinkClass{ delay: 2 }],
launch_configurations: [
  Slurm{ job_pack_size: 1, time: "20-00:00:00" }]]}

```